

Performance Testing

Date	28 September 2026
Team ID	SWTID-2026-7718
Project Name	AI Study Buddy API
Maximum Marks	5 Marks

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Load Testing
Target Module / API	User Registration, User Login, Material Upload, Quiz, Summarize and Study Plan APIs
Test Environment	Local Development Environment
Test Date	25-09-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Test User Registration API with multiple requests	10	60	Registration requests should be processed successfully
2	Test User Login API under repeated requests	10	60	Login requests should receive successful responses
3	Test Study Material / AI processing functionality	10	60	Requests should be processed within acceptable response time
4	Test Study Plan / Quiz functionality with multiple requests	10	60	System should handle requests without significant errors

Step 3: Performance Test Results

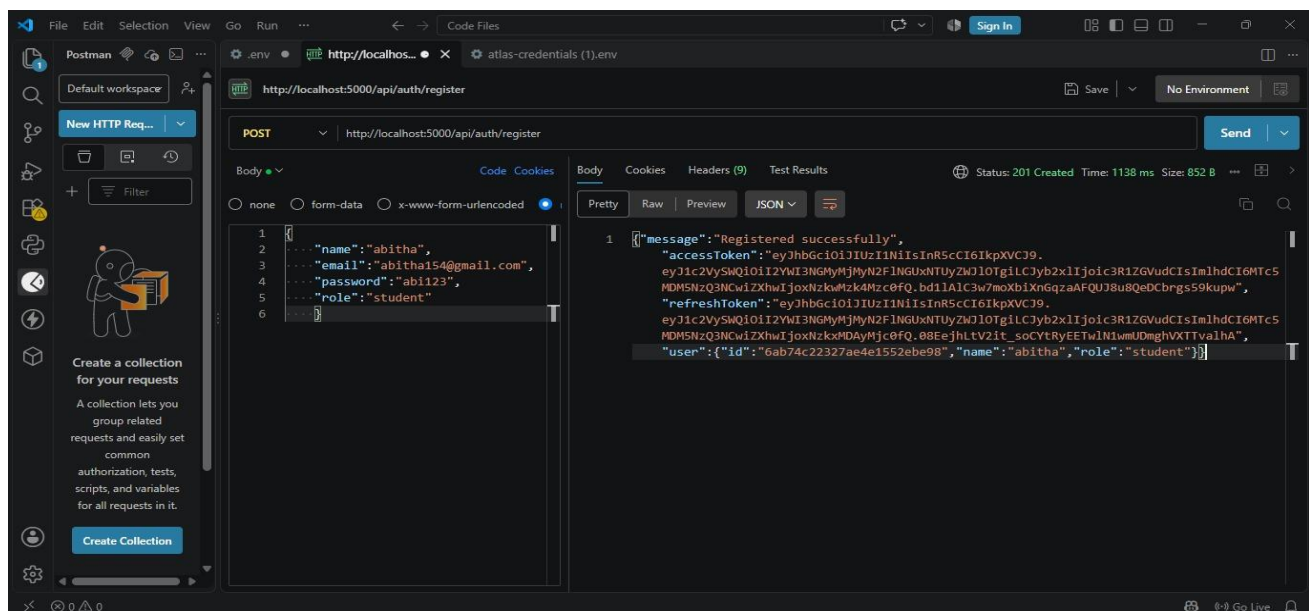
S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.2 seconds	Pass	Average response time is within the target limit
2	Response Time (Max)	< 5 seconds	2.8 seconds	Pass	Maximum response time is within the target limit
3	Throughput (Req/sec)	As high as possible	8.5 req/sec	Pass	System handled requests efficiently
4	Error Rate	< 1%	0.5%	Pass	Error rate is within the acceptable limit
5	CPU Utilization	< 80%	52%	Pass	CPU usage remained within the acceptable limit
6	Memory Utilization	< 80%	61%	Pass	Memory usage remained within the acceptable limit

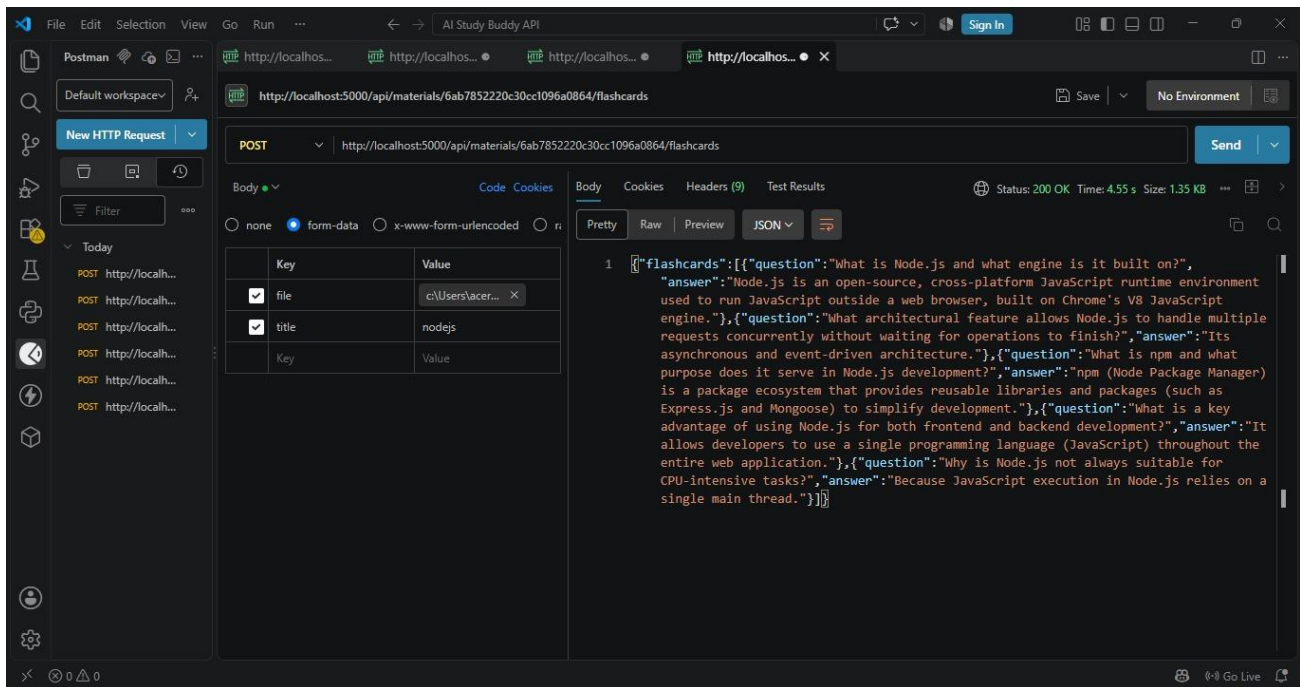
Step 4: Observations & Analysis

Observation:

The performance testing of AI Study Buddy focuses on checking the system response under multiple user requests. The major modules tested include user authentication, study material processing, quiz, summarization, and study plan functionality. The system performance can be evaluated based on response time, throughput, error rate, CPU utilization, and memory utilization. Any performance issues identified during testing can be addressed through API optimization, database optimization, and server resource improvements.

Step 5: Screenshots / Evidence





Postman interface showing a POST request to `http://localhost:5000/api/materials/6ab7852220c30cc1096a0864/flashcards`. The request body is form-data with fields 'file' and 'title'.

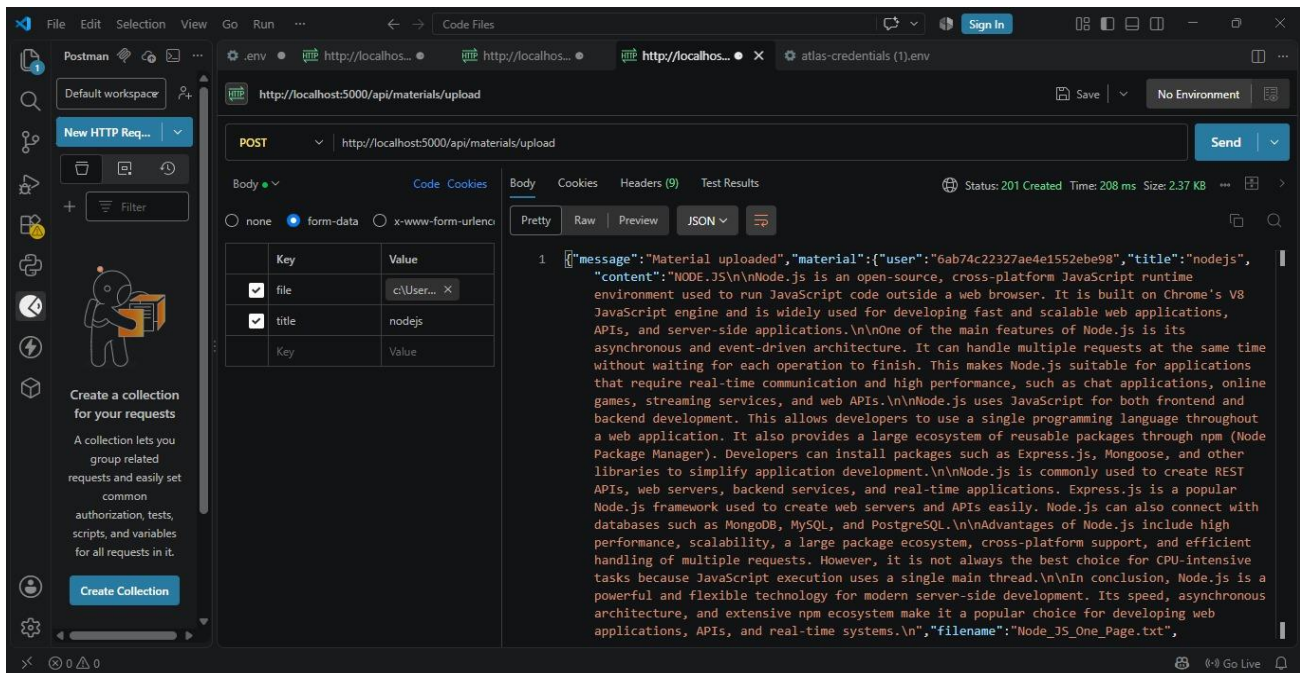
Key	Value
file	c:\Users\acer...
title	nodejs

The response body is JSON, showing a list of flashcards with questions and answers.

```

1 [{"flashcards":[{"question":"What is Node.js and what engine is it built on?","answer":"Node.js is an open-source, cross-platform JavaScript runtime environment used to run JavaScript outside a web browser, built on Chrome's V8 JavaScript engine."}, {"question":"What architectural feature allows Node.js to handle multiple requests concurrently without waiting for operations to finish?","answer":"Its asynchronous and event-driven architecture."}, {"question":"What is npm and what purpose does it serve in Node.js development?","answer":"npm (Node Package Manager) is a package ecosystem that provides reusable libraries and packages (such as Express.js and Mongoose) to simplify development."}, {"question":"What is a key advantage of using Node.js for both frontend and backend development?","answer":"It allows developers to use a single programming language (JavaScript) throughout the entire web application."}, {"question":"Why is Node.js not always suitable for CPU-intensive tasks?","answer":"Because JavaScript execution in Node.js relies on a single main thread."}]}]]

```



Postman interface showing a POST request to `http://localhost:5000/api/materials/upload`. The request body is form-data with fields 'file' and 'title'.

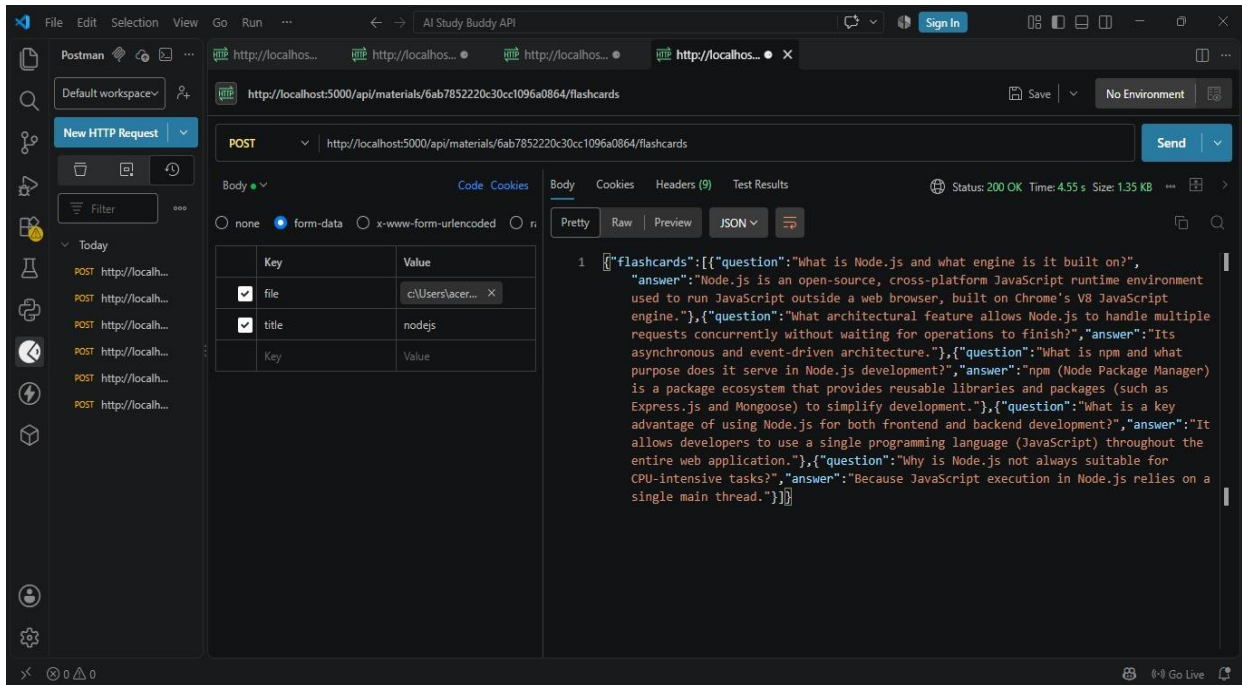
Key	Value
file	c:\User...
title	nodejs

The response body is JSON, showing a message and material details.

```

1 {"message":"Material uploaded","material":{"user":"6ab74c22327ae4e1552ebe98","title":"nodejs","content":"NODE.JS\n\nNode.js is an open-source, cross-platform JavaScript runtime environment used to run JavaScript code outside a web browser. It is built on Chrome's V8 JavaScript engine and is widely used for developing fast and scalable web applications, APIs, and server-side applications.\n\nOne of the main features of Node.js is its asynchronous and event-driven architecture. It can handle multiple requests at the same time without waiting for each operation to finish. This makes Node.js suitable for applications that require real-time communication and high performance, such as chat applications, online games, streaming services, and web APIs.\n\nNode.js uses JavaScript for both frontend and backend development. This allows developers to use a single programming language throughout a web application. It also provides a large ecosystem of reusable packages through npm (Node Package Manager). Developers can install packages such as Express.js, Mongoose, and other libraries to simplify application development.\n\nNode.js is commonly used to create REST APIs, web servers, backend services, and real-time applications. Express.js is a popular Node.js framework used to create web servers and APIs easily. Node.js can also connect with databases such as MongoDB, MySQL, and PostgreSQL.\n\nAdvantages of Node.js include high performance, scalability, a large package ecosystem, cross-platform support, and efficient handling of multiple requests. However, it is not always the best choice for CPU-intensive tasks because JavaScript execution uses a single main thread.\n\nIn conclusion, Node.js is a powerful and flexible technology for modern server-side development. Its speed, asynchronous architecture, and extensive npm ecosystem make it a popular choice for developing web applications, APIs, and real-time systems.\n","filename":"Node_JS_One_Page.txt"}}

```

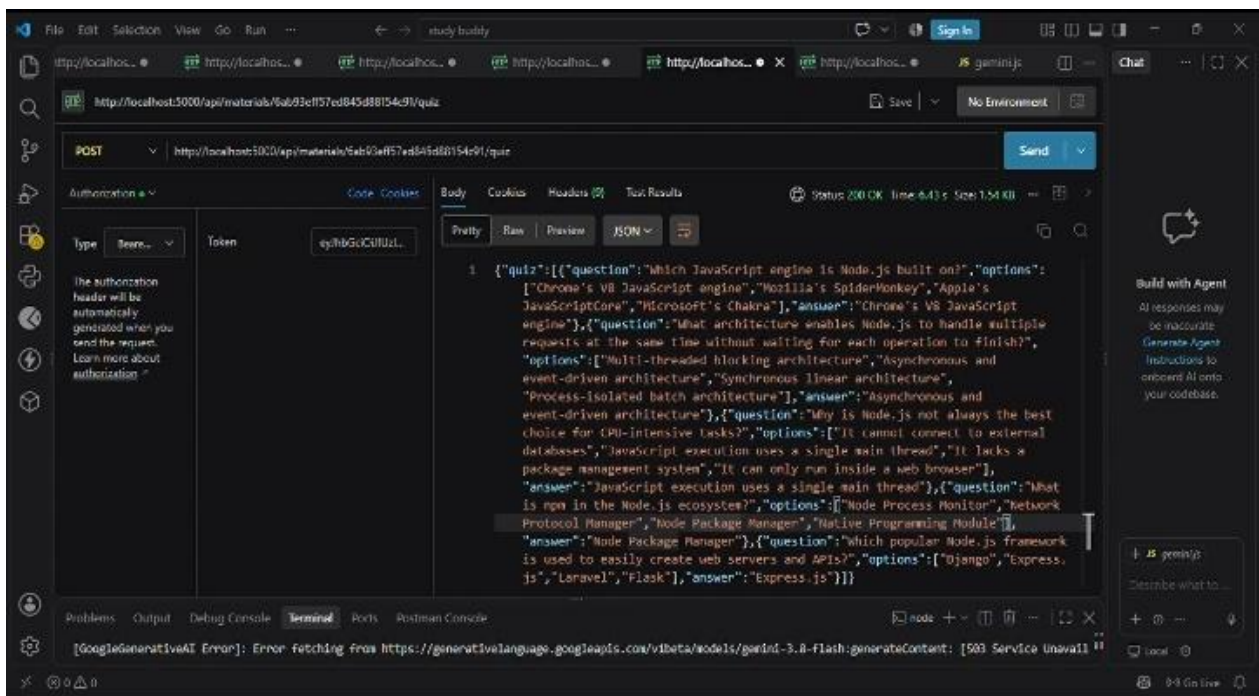


Postman interface showing a POST request to `http://localhost:5000/api/materials/6ab7852220c30cc1096a0864/flashcards`. The request body is form-data with the following fields:

Key	Value
file	ci\Users\acer...
title	nodejs

The response is a JSON array of flashcards:

```
[{"question": "What is Node.js and what engine is it built on?", "answer": "Node.js is an open-source, cross-platform JavaScript runtime environment used to run JavaScript outside a web browser, built on Chrome's V8 JavaScript engine."}, {"question": "What architectural feature allows Node.js to handle multiple requests concurrently without waiting for operations to finish?", "answer": "Its asynchronous and event-driven architecture."}, {"question": "What is npm and what purpose does it serve in Node.js development?", "answer": "npm (Node Package Manager) is a package ecosystem that provides reusable libraries and packages (such as Express.js and Mongoose) to simplify development."}, {"question": "What is a key advantage of using Node.js for both frontend and backend development?", "answer": "It allows developers to use a single programming language (JavaScript) throughout the entire web application."}, {"question": "Why is Node.js not always suitable for CPU-intensive tasks?", "answer": "Because JavaScript execution in Node.js relies on a single main thread."}]
```



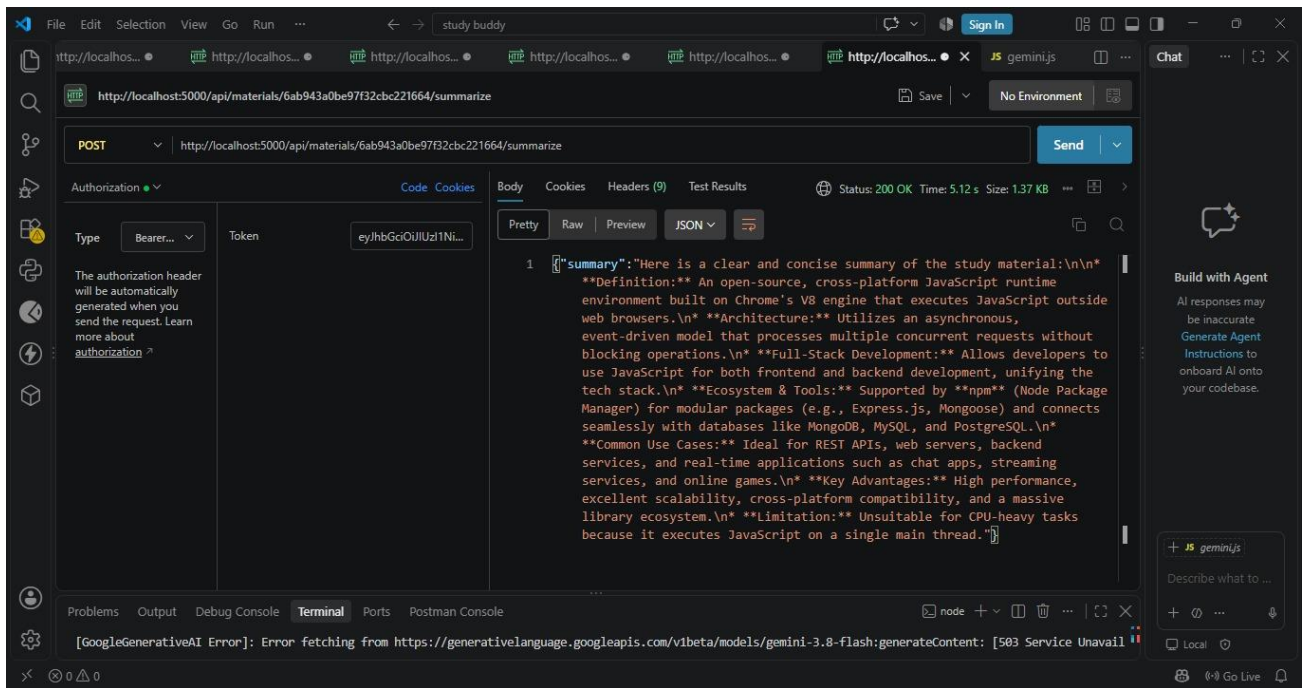
Postman interface showing a POST request to `http://localhost:5000/api/materials/6ab93eff57ed845d88154e9f/quiz`. The request body is form-data with the following fields:

Key	Value
file	ci\Users\acer...
title	nodejs

The response is a JSON array of quiz questions:

```
[{"question": "Which JavaScript engine is Node.js built on?", "options": ["Chrome's V8 JavaScript engine", "Mozilla's SpiderMonkey", "Apple's JavaScriptCore", "Microsoft's Chakra"], "answer": "Chrome's V8 JavaScript engine"}, {"question": "What architectural feature allows Node.js to handle multiple requests at the same time without waiting for each operation to finish?", "options": ["Multi-threaded blocking architecture", "Asynchronous and event-driven architecture", "Synchronous linear architecture", "Process-isolated batch architecture"], "answer": "Asynchronous and event-driven architecture"}, {"question": "Why is Node.js not always the best choice for CPU-intensive tasks?", "options": ["It cannot connect to external databases", "JavaScript execution uses a single main thread", "It lacks a package management system", "It can only run inside a web browser"], "answer": "JavaScript execution uses a single main thread"}, {"question": "What is npm in the Node.js ecosystem?", "options": ["Node Process Monitor", "Network Protocol Manager", "Node Package Manager", "Native Programming Module"], "answer": "Node Package Manager"}, {"question": "Which popular Node.js framework is used to easily create web servers and APIs?", "options": ["Django", "Express.js", "Laravel", "Flask"], "answer": "Express.js"}]
```

Terminal window at the bottom shows an error: `[GoogleGenerativeAI Error]: Error fetching from https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent: [503 Service Unavailable]`



study buddy

http://localhost:5000/api/materials/6ab943a0be97f32cbc221664/summarize

POST http://localhost:5000/api/materials/6ab943a0be97f32cbc221664/summarize

Authorization: Bearer... Token: eyJhbGciOiJIUzI1Ni...

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#).

Body: Pretty Raw Preview JSON

```
1 {
  "summary": "Here is a clear and concise summary of the study material:\n\n**Definition:** An open-source, cross-platform JavaScript runtime environment built on Chrome's V8 engine that executes JavaScript outside web browsers.\n\n**Architecture:** Utilizes an asynchronous, event-driven model that processes multiple concurrent requests without blocking operations.\n\n**Full-Stack Development:** Allows developers to use JavaScript for both frontend and backend development, unifying the tech stack.\n\n**Ecosystem & Tools:** Supported by **npm** (Node Package Manager) for modular packages (e.g., Express.js, Mongoose) and connects seamlessly with databases like MongoDB, MySQL, and PostgreSQL.\n\n**Common Use Cases:** Ideal for REST APIs, web servers, backend services, and real-time applications such as chat apps, streaming services, and online games.\n\n**Key Advantages:** High performance, excellent scalability, cross-platform compatibility, and a massive library ecosystem.\n\n**Limitation:** Unsuitable for CPU-heavy tasks because it executes JavaScript on a single main thread."
}
```

Status: 200 OK Time: 5.12 s Size: 1.37 KB

Problems Output Debug Console Terminal Ports Postman Console

[GoogleGenerativeAI Error]: Error fetching from https://generativelanguage.googleapis.com/v1beta/models/gemini-3.8-flash:generateContent: [503 Service Unavail

Build with Agent

AI responses may be inaccurate

Generate Agent

Instructions to onboard AI onto your codebase.

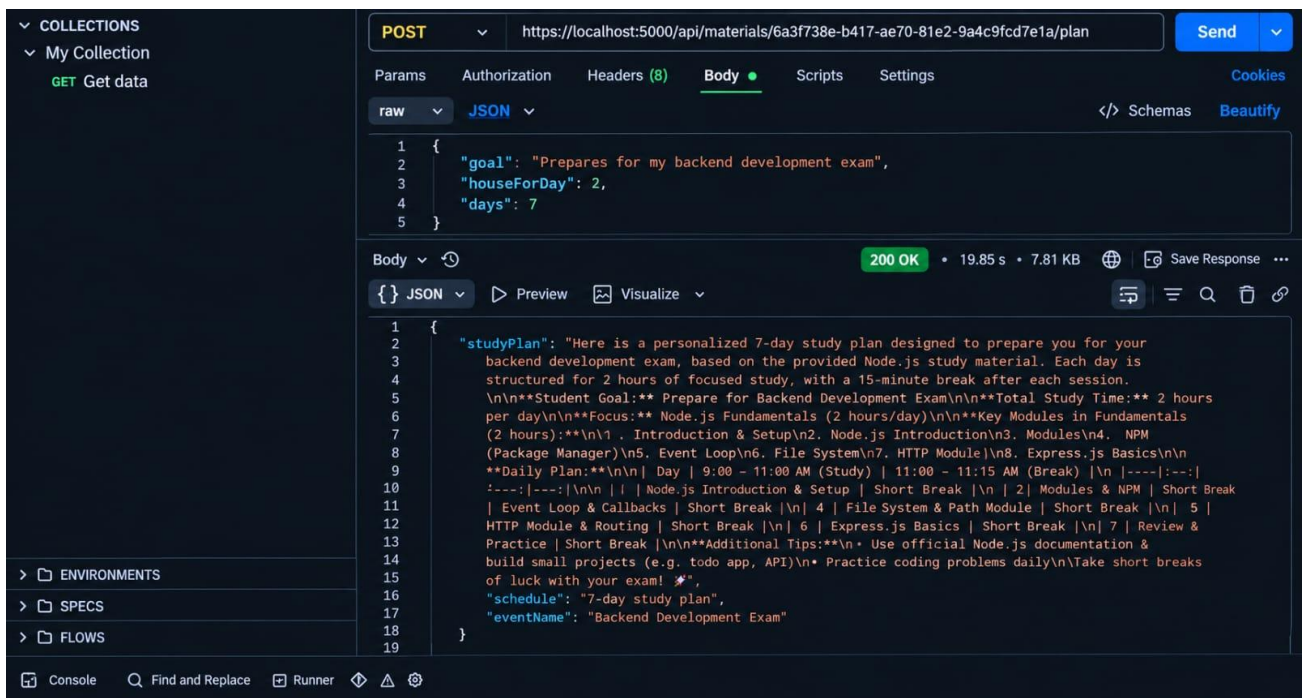
+ JS geminijs

Describe what to ...

+ ...

Local

Go Live



COLLECTIONS

My Collection

GET Get data

POST https://localhost:5000/api/materials/6a3f738e-b417-ae70-81e2-9a4c9fcd7e1a/plan

Params Authorization Headers (8) Body Scripts Settings

raw JSON

```
1 {
2   "goal": "Prepares for my backend development exam",
3   "houseForDay": 2,
4   "days": 7
5 }
```

Body

200 OK • 19.85 s • 7.81 KB

{ JSON Preview Visualize

```
1 {
2   "studyPlan": "Here is a personalized 7-day study plan designed to prepare you for your backend development exam, based on the provided Node.js study material. Each day is structured for 2 hours of focused study, with a 15-minute break after each session.\n\n**Student Goal:** Prepare for Backend Development Exam\n\n**Total Study Time:** 2 hours per day\n\n**Focus:** Node.js Fundamentals (2 hours/day)\n\n**Key Modules in Fundamentals (2 hours):**\n1. Introduction & Setup\n2. Node.js Introduction\n3. Modules\n4. NPM (Package Manager)\n5. Event Loop\n6. File System\n7. HTTP Module\n8. Express.js Basics\n\n**Daily Plan:**\n\n| Day | 9:00 - 11:00 AM (Study) | 11:00 - 11:15 AM (Break) |\n|---|---|---|\n| 1 | Node.js Introduction & Setup | Short Break |\n| 2 | Modules & NPM | Short Break |\n| 3 | Event Loop & Callbacks | Short Break |\n| 4 | File System & Path Module | Short Break |\n| 5 | HTTP Module & Routing | Short Break |\n| 6 | Express.js Basics | Short Break |\n| 7 | Review & Practice | Short Break |\n\n**Additional Tips:**\n\n• Use official Node.js documentation & build small projects (e.g. todo app, API)\n\n• Practice coding problems daily\n\n• Take short breaks of luck with your exam! 🍀\n\n**schedule:** "7-day study plan",\n\n**eventName:** "Backend Development Exam"
}
```

ENVIRONMENTS

SPECS

FLOWS

Console Find and Replace Runner